DUMMY MODEL

Results for ‘most frequent’:

- Test stratified: accuracy=0.460, macro-F1=0.057
- Test n_rules_out: accuracy=0.350, macro-F1=0.086
- Test n_communities_out: accuracy=0.085, macro-F1=0.031

Results for ‘uniform’:

- Test stratified: accuracy=0.090, macro-F1=0.051
- Test n_rules_out: accuracy=0.067, macro-F1=0.041
- Test n_communities_out: accuracy=0.081, macro-F1=0.037

All dummy baselines perform extremely poorly. So basically it confirms that task is non-trivial and not solvable by frequency or random-based strategies.

And our model works much better than just random guessing.

PARAMETERS THAT WERE CONSIDERED

For learning rate:

- 5.e-06
- 1.e-05
- 2.e-05

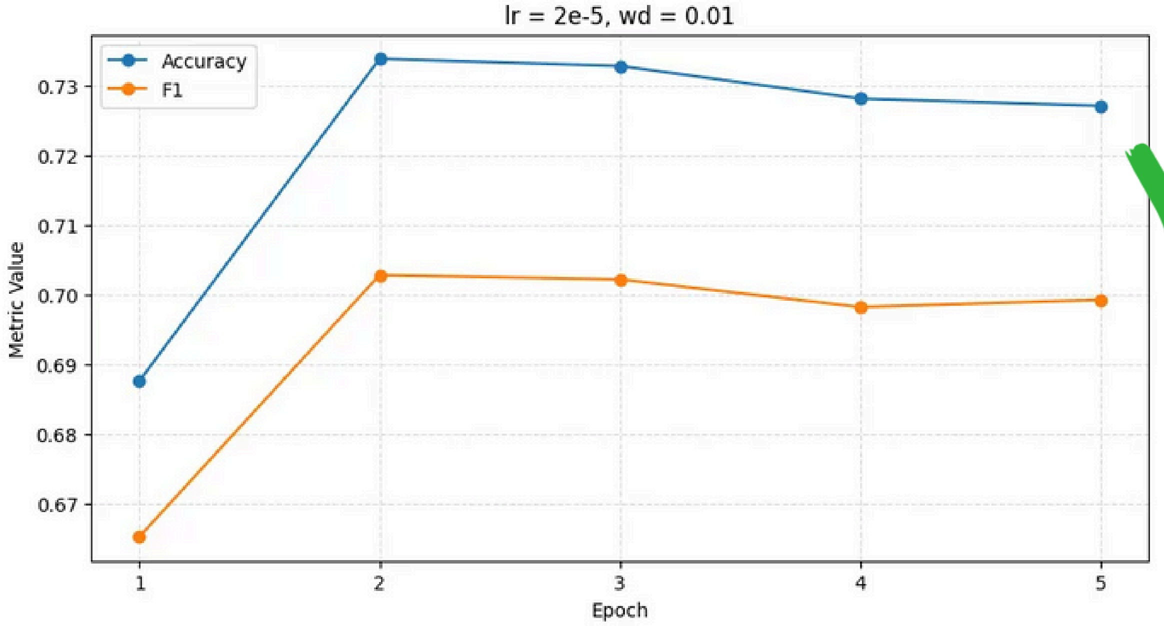
For weight decays:

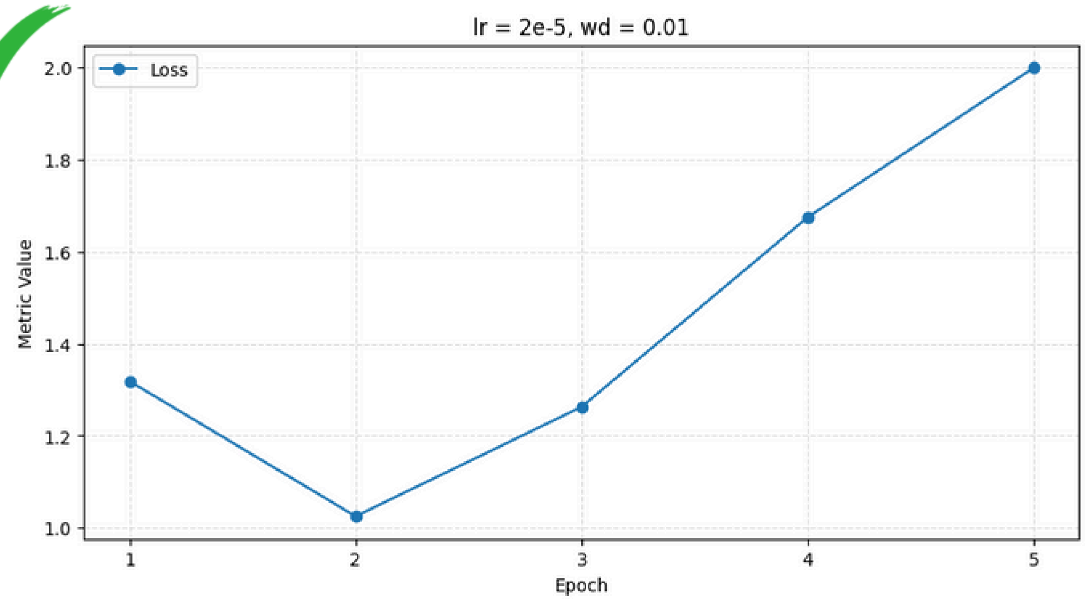
- 0.1
- 0.01
- 0.001
- 0.02
- 0.03

My weight decay selection was iterative: I evaluated how each value performed with the current learning rate and adjusted the next one to uncover meaningful trends.

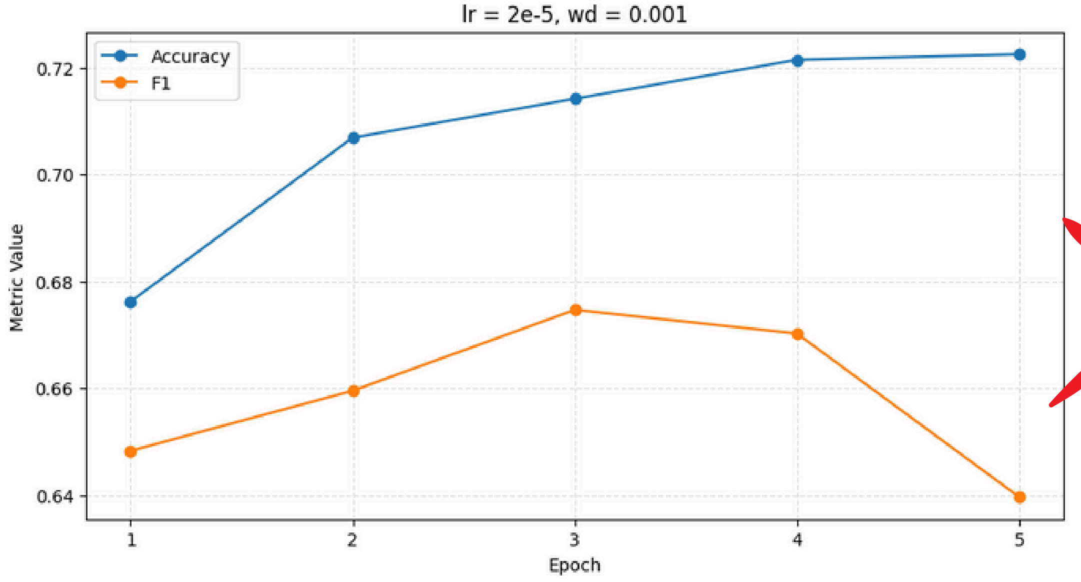
FIRST I ANALYSED HOW THEY PERFORM DURING THE TRAINING. ->

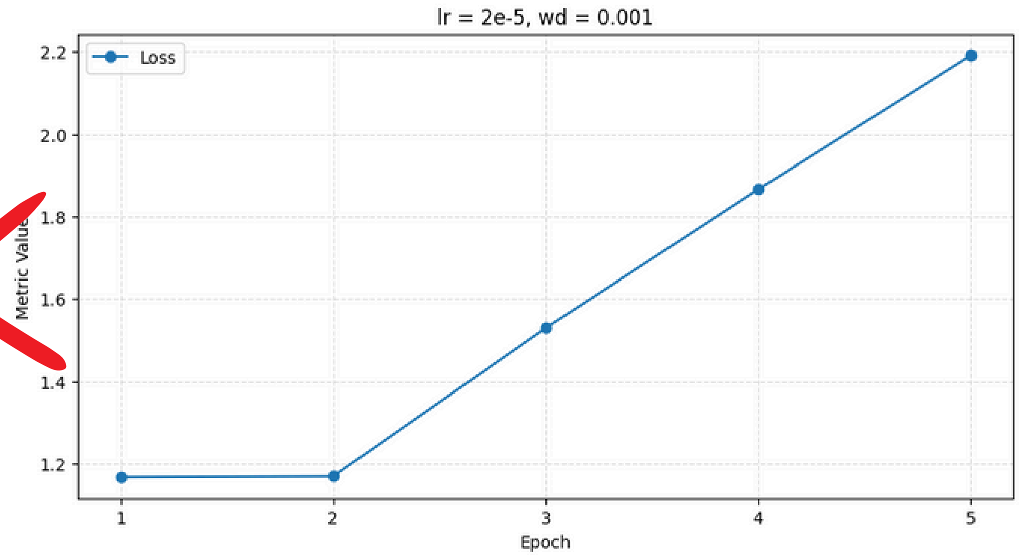
LR=2E-05

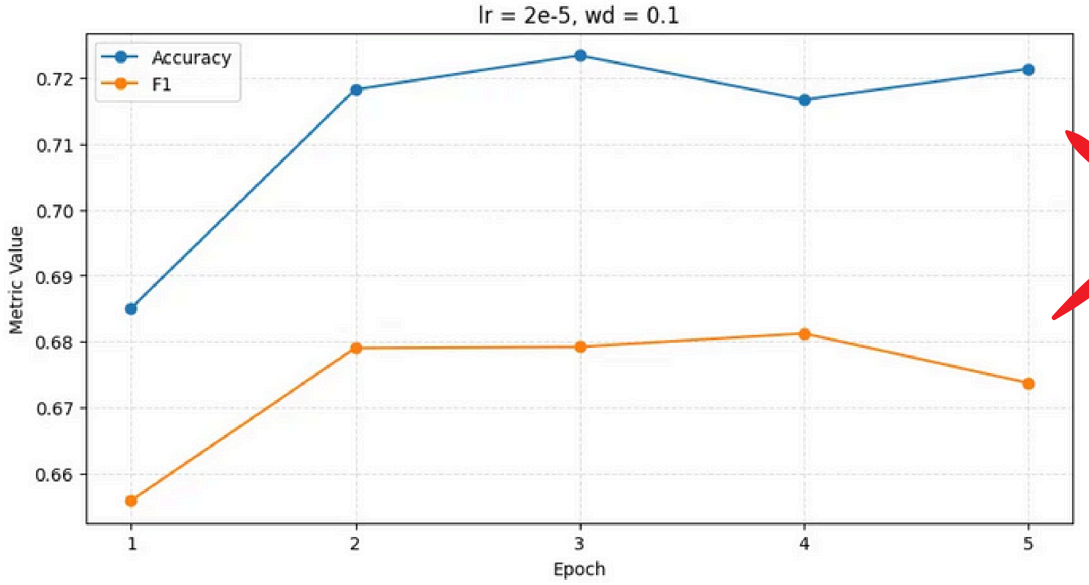


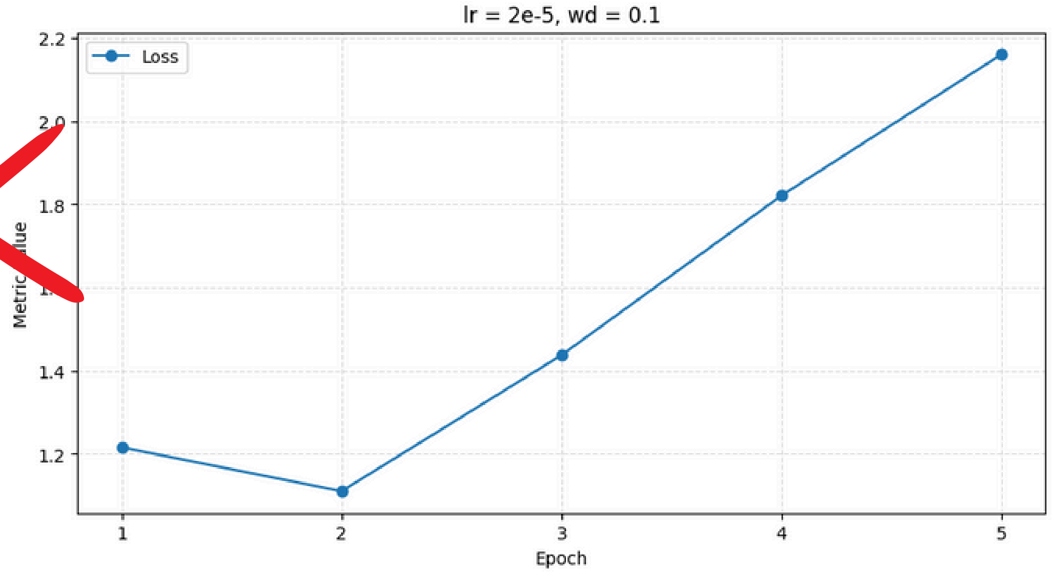


this combination shows the most stable performance on every metrics in comparison to other 2

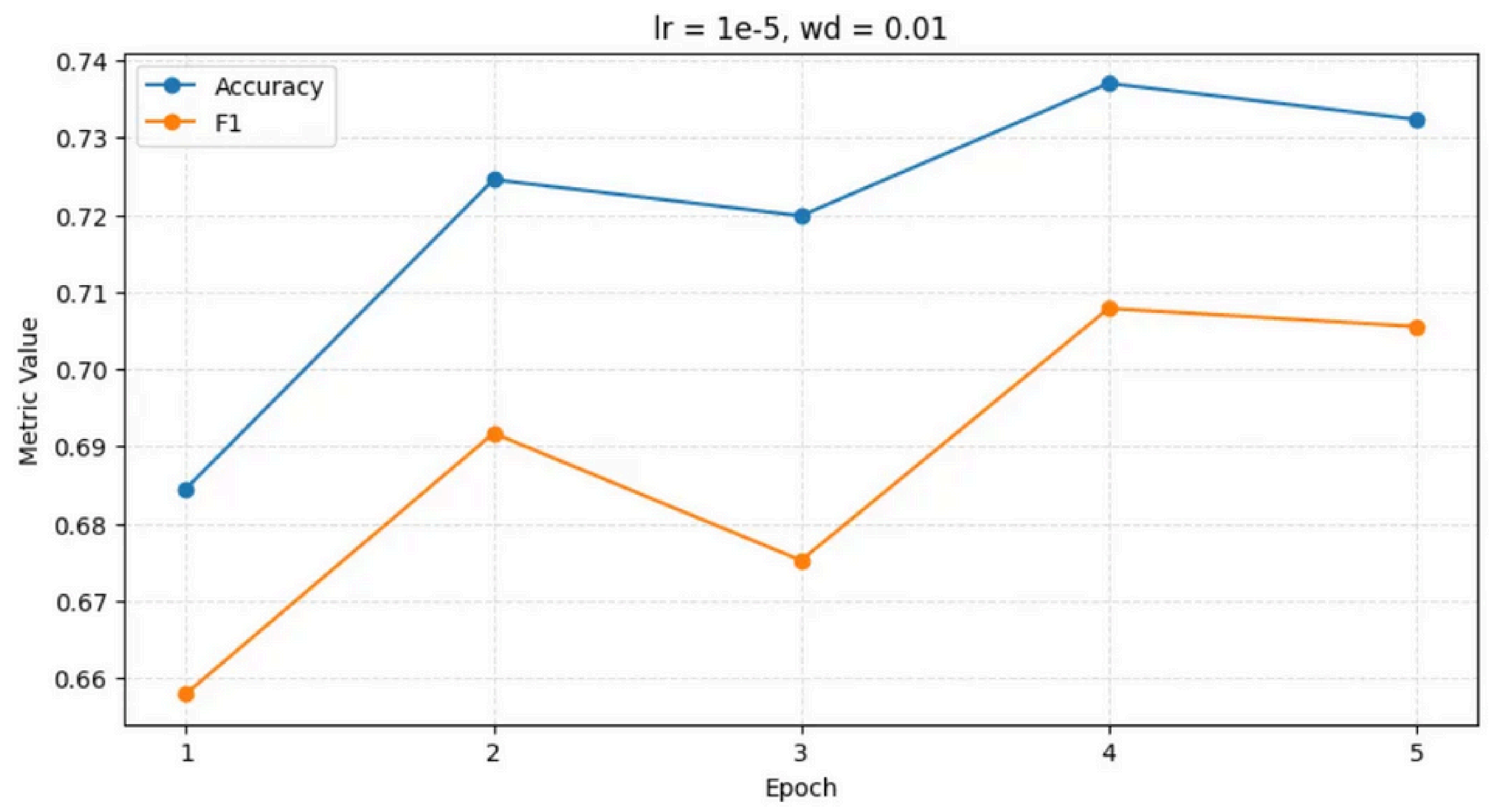


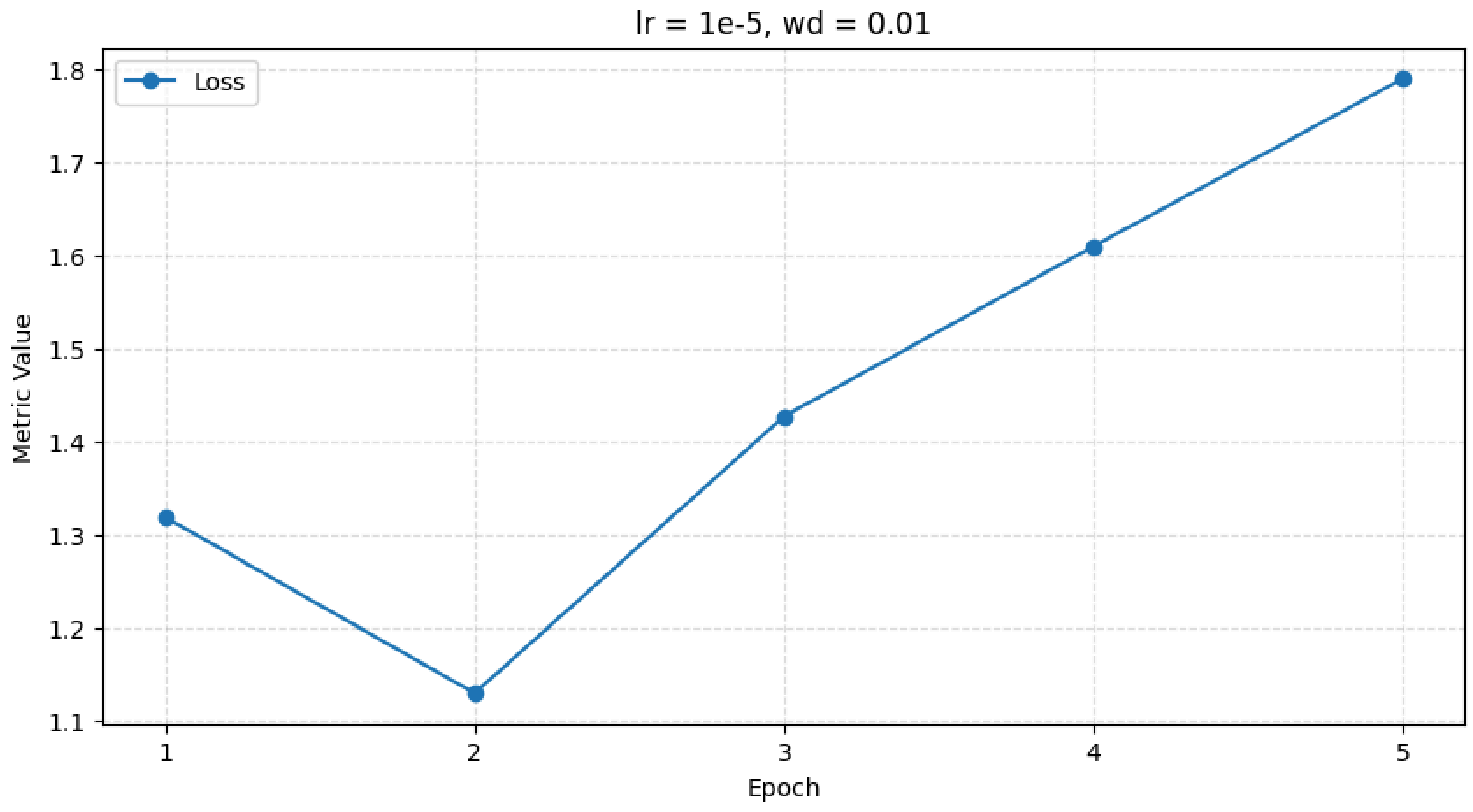






LR=1E-05

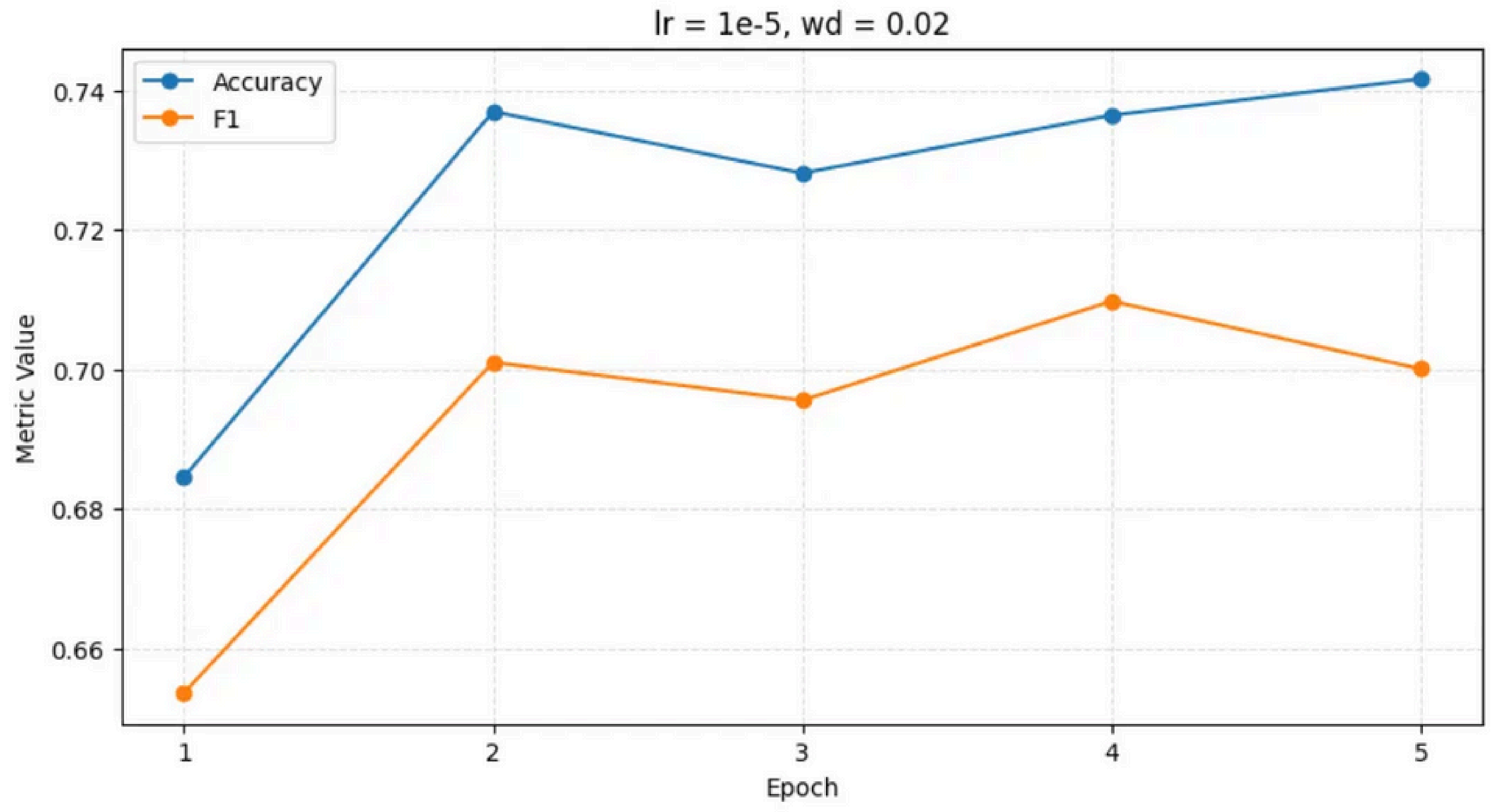


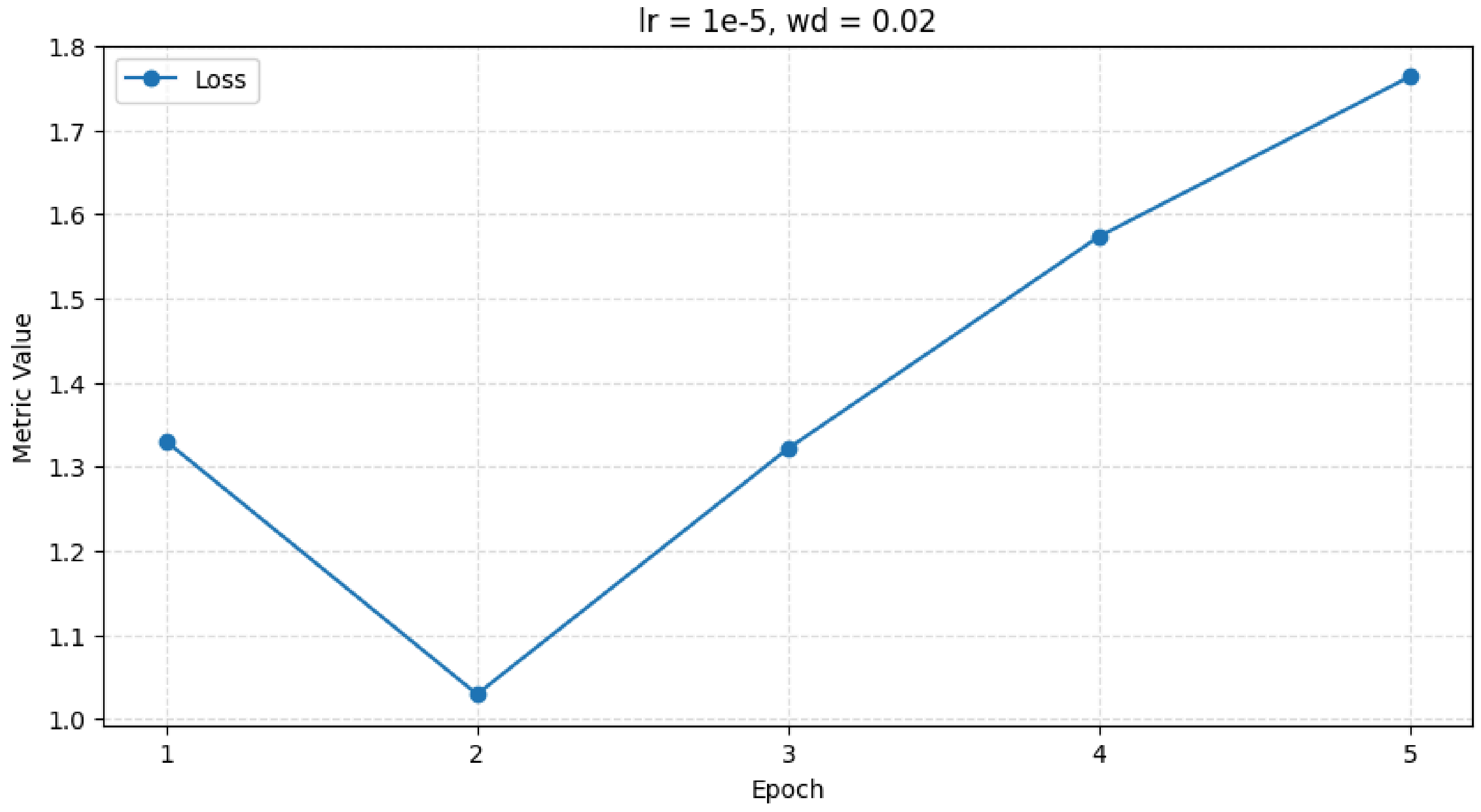


Here the loss is way smaller than for lr=2r-05 and it does not increase that quickly. Also f1 is bigger.

So I decided to check for slightly bigger wd to see if it gets better

LR=1E-05

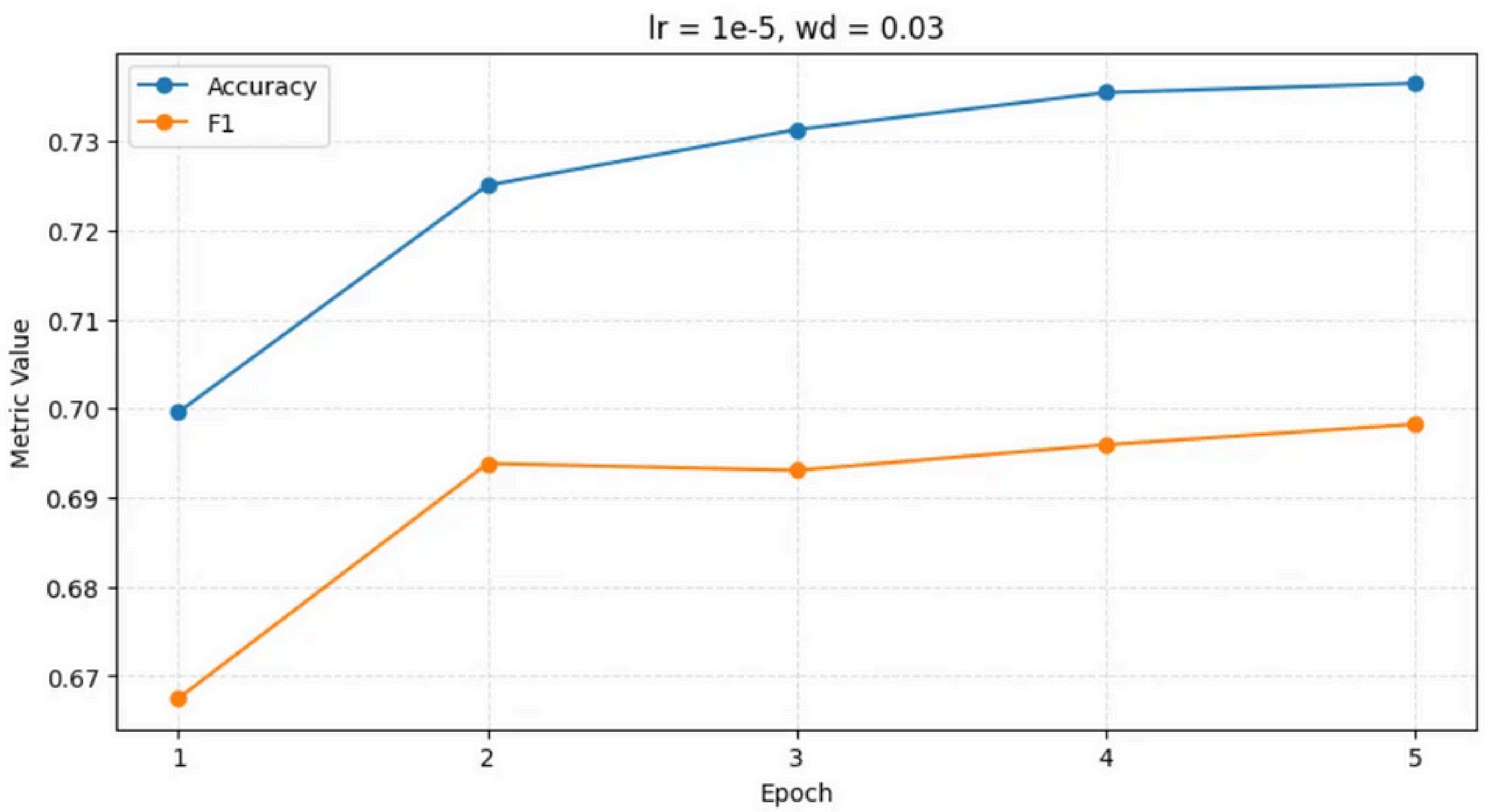


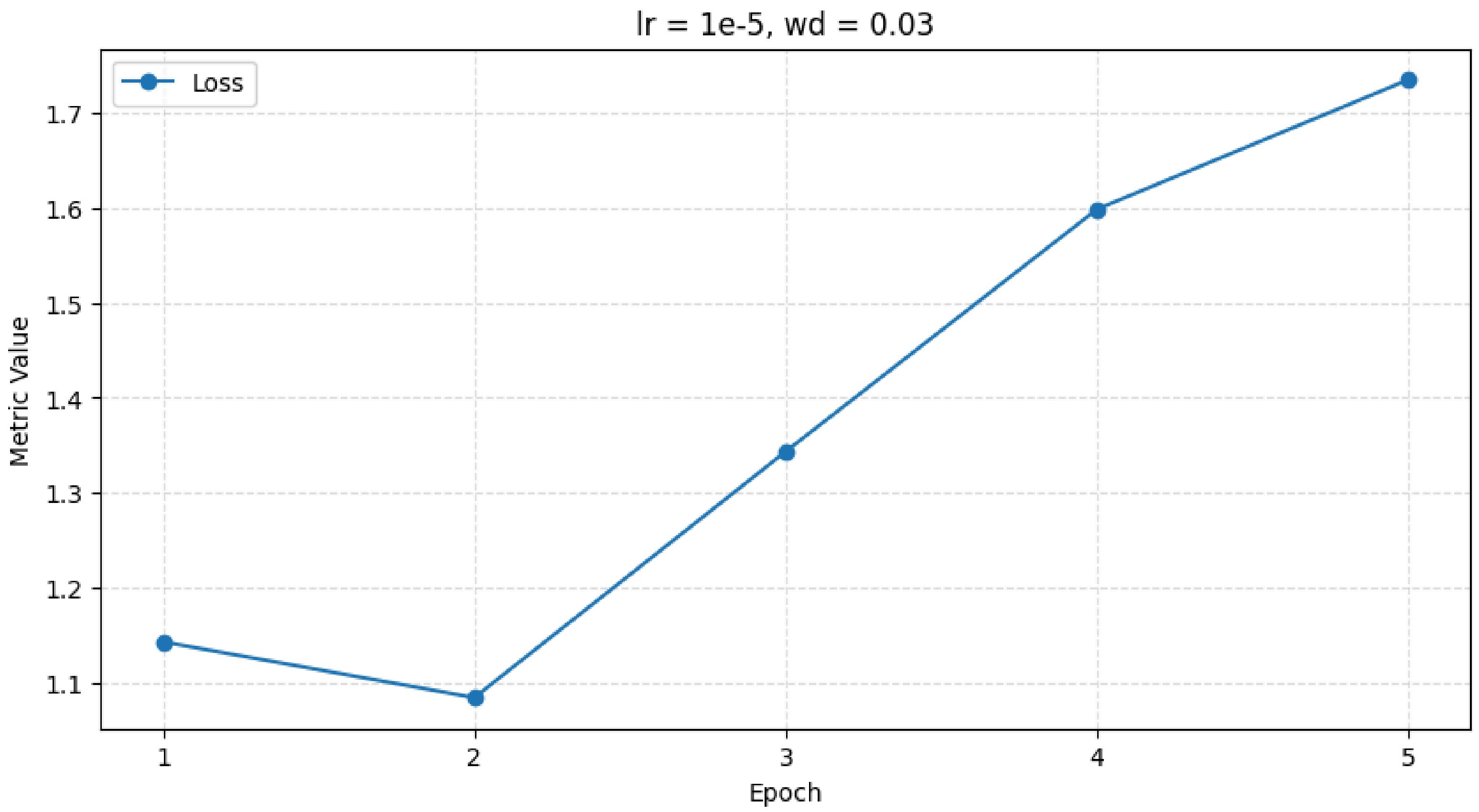


The loss becomes smaller and performance seems to get better.

Let’s check if it gets even better with 0.03

LR=1E-05

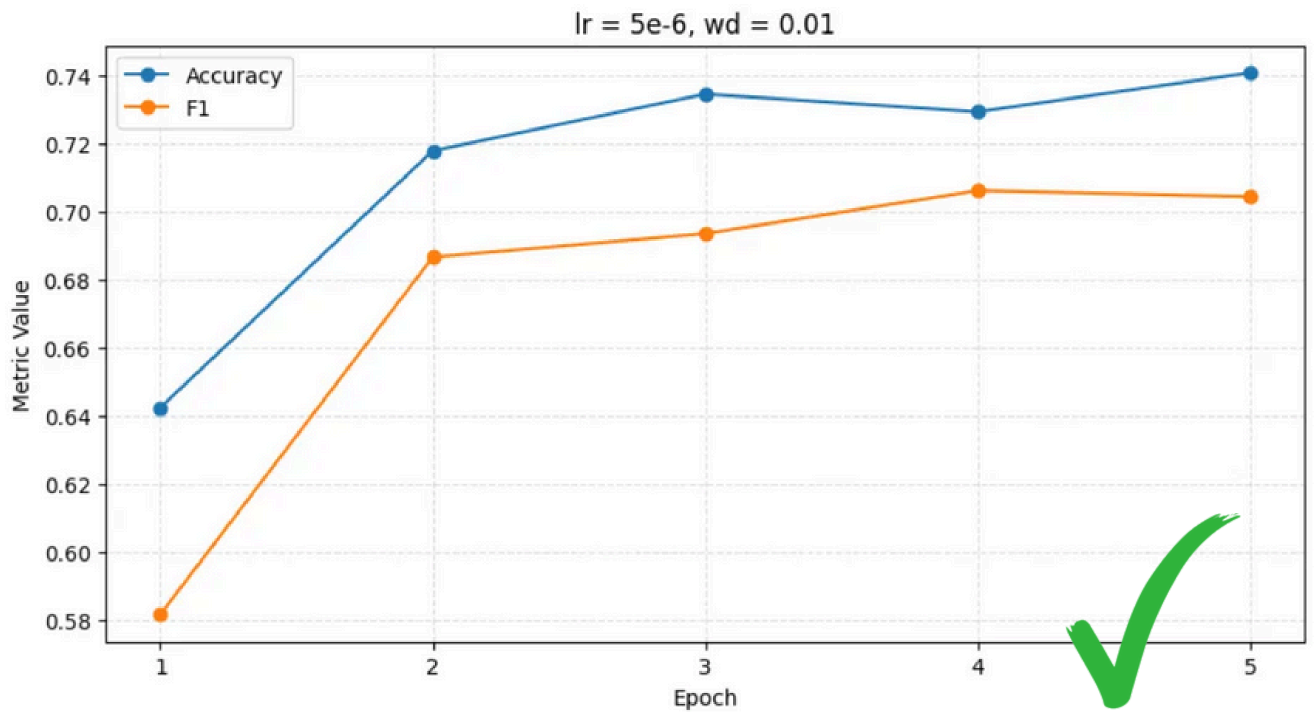


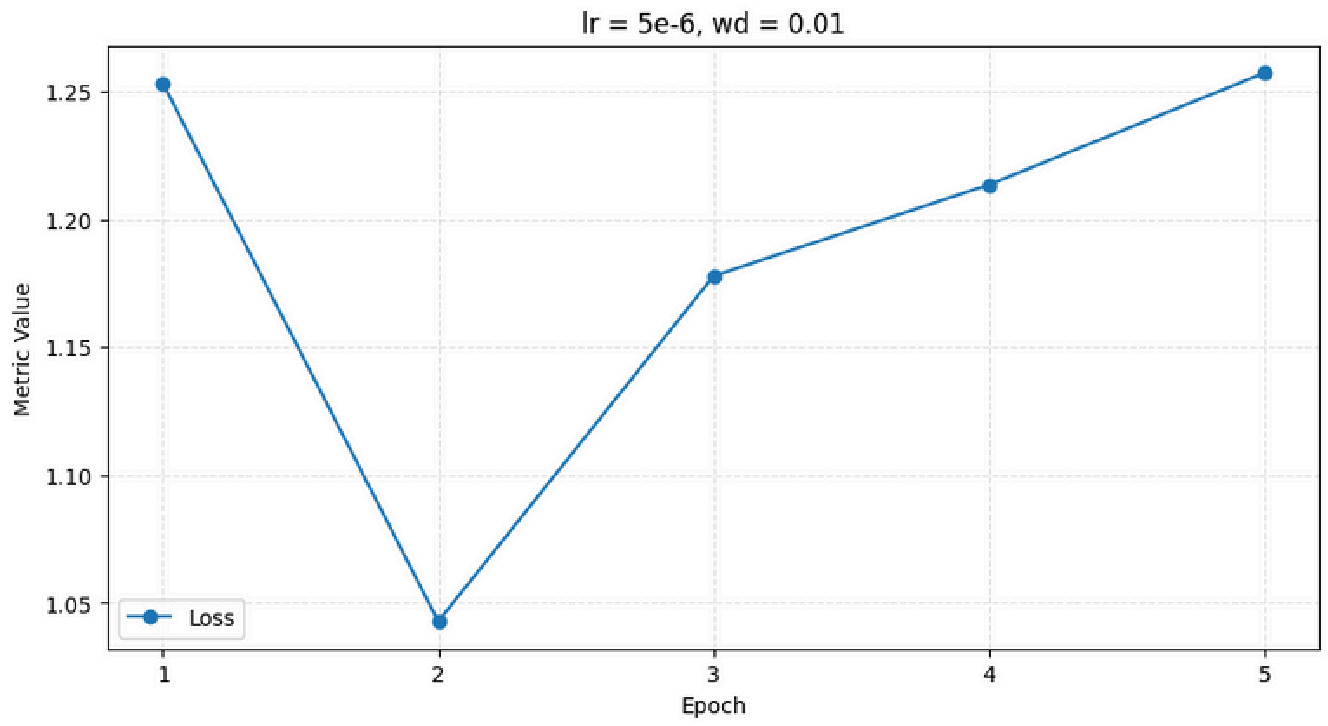


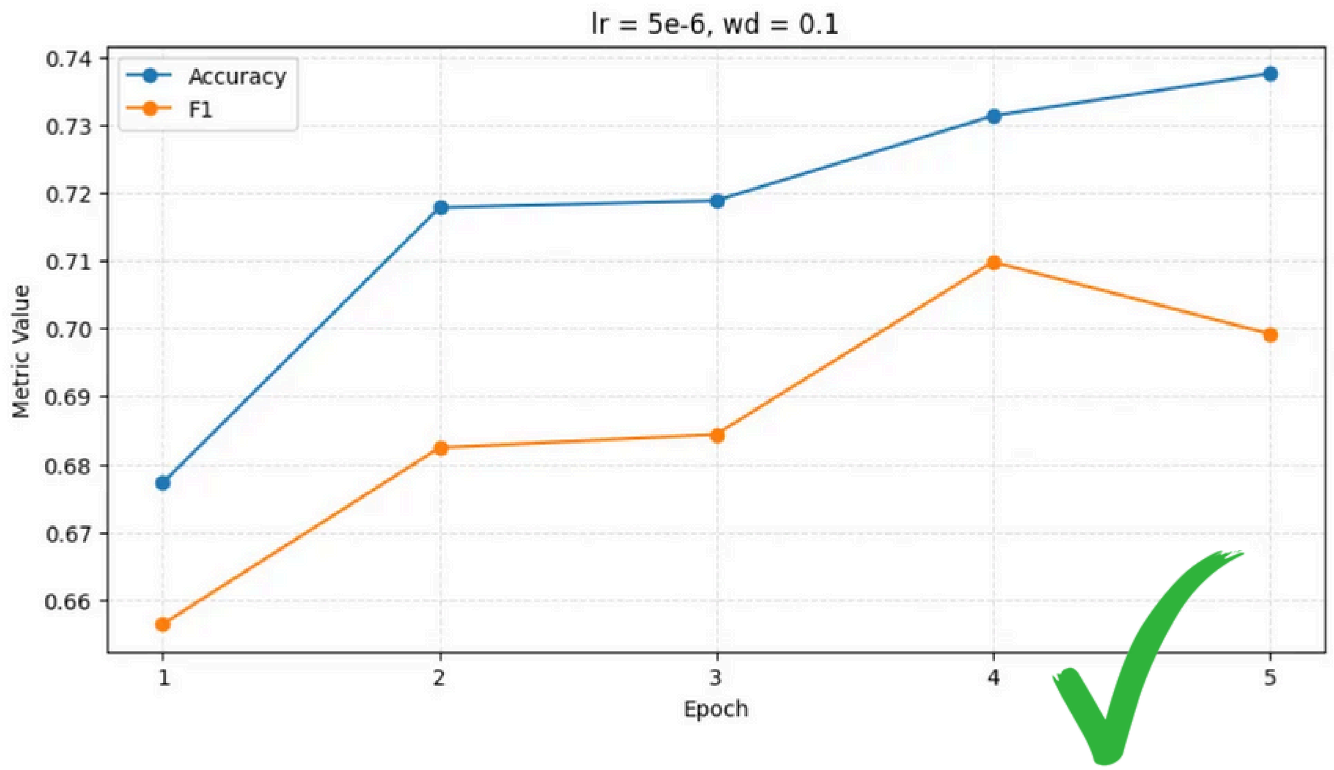
The loss became indeed smaller and performance more stable.

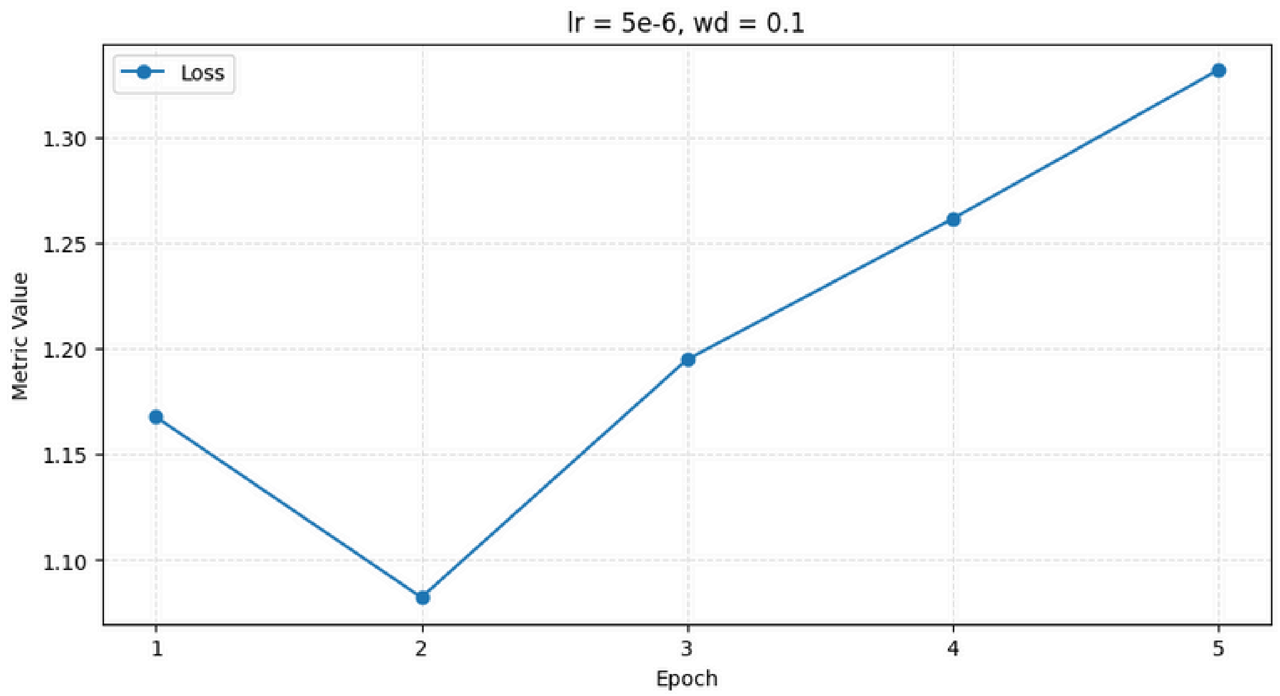
I think lr = 1e-05 and wd = 0.02/wd=0.03 can be considered as reasonable option

LR = 5E-6



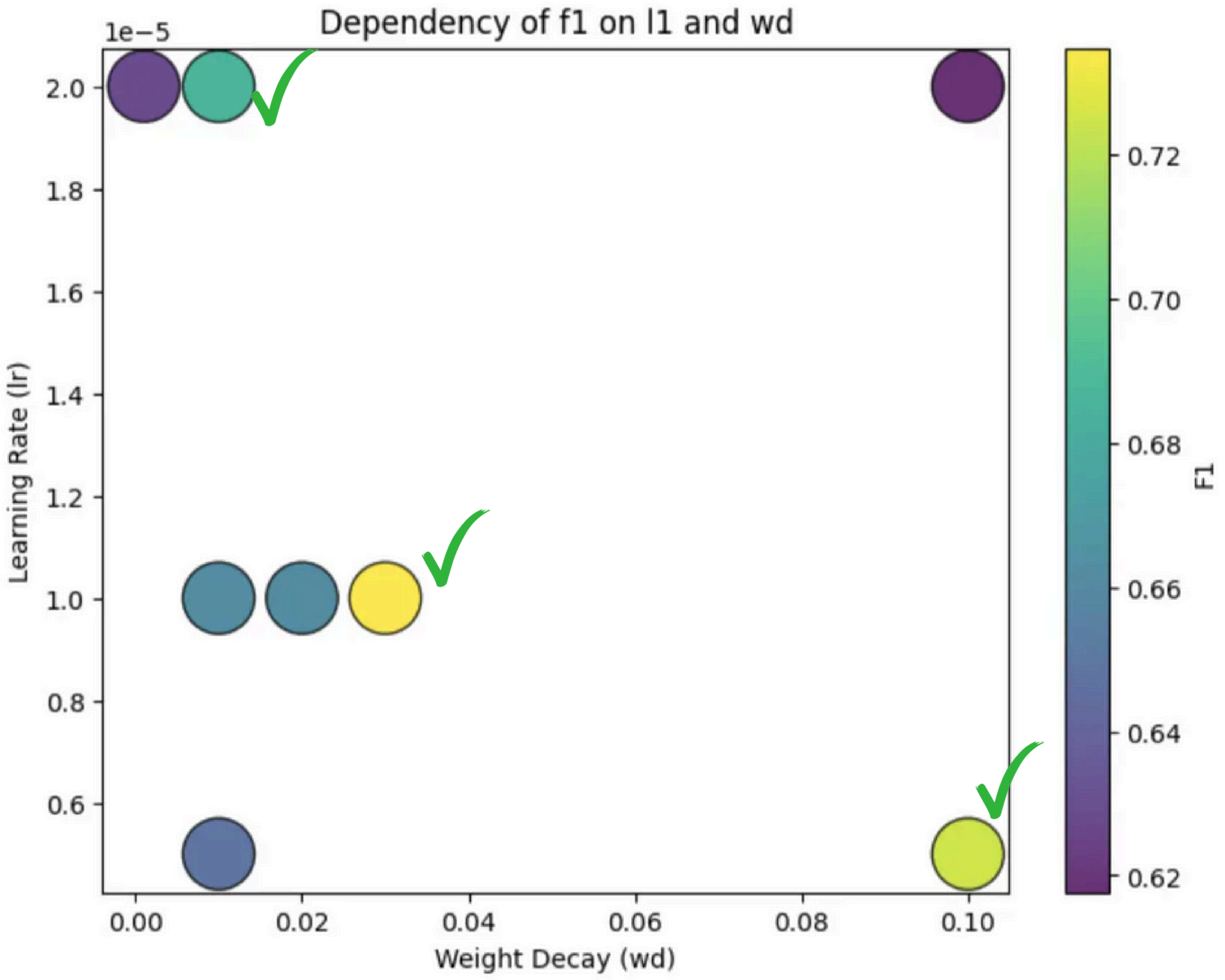






They are kinda similar, but wd = 0.01 gives a little bit better performance. Though both of them can be considered further.

EVENTUALLY I TESTED ON STRATIFIED TEST DATASET



So based on stratified testing results I limited the scope of suitable parameter combinations within:

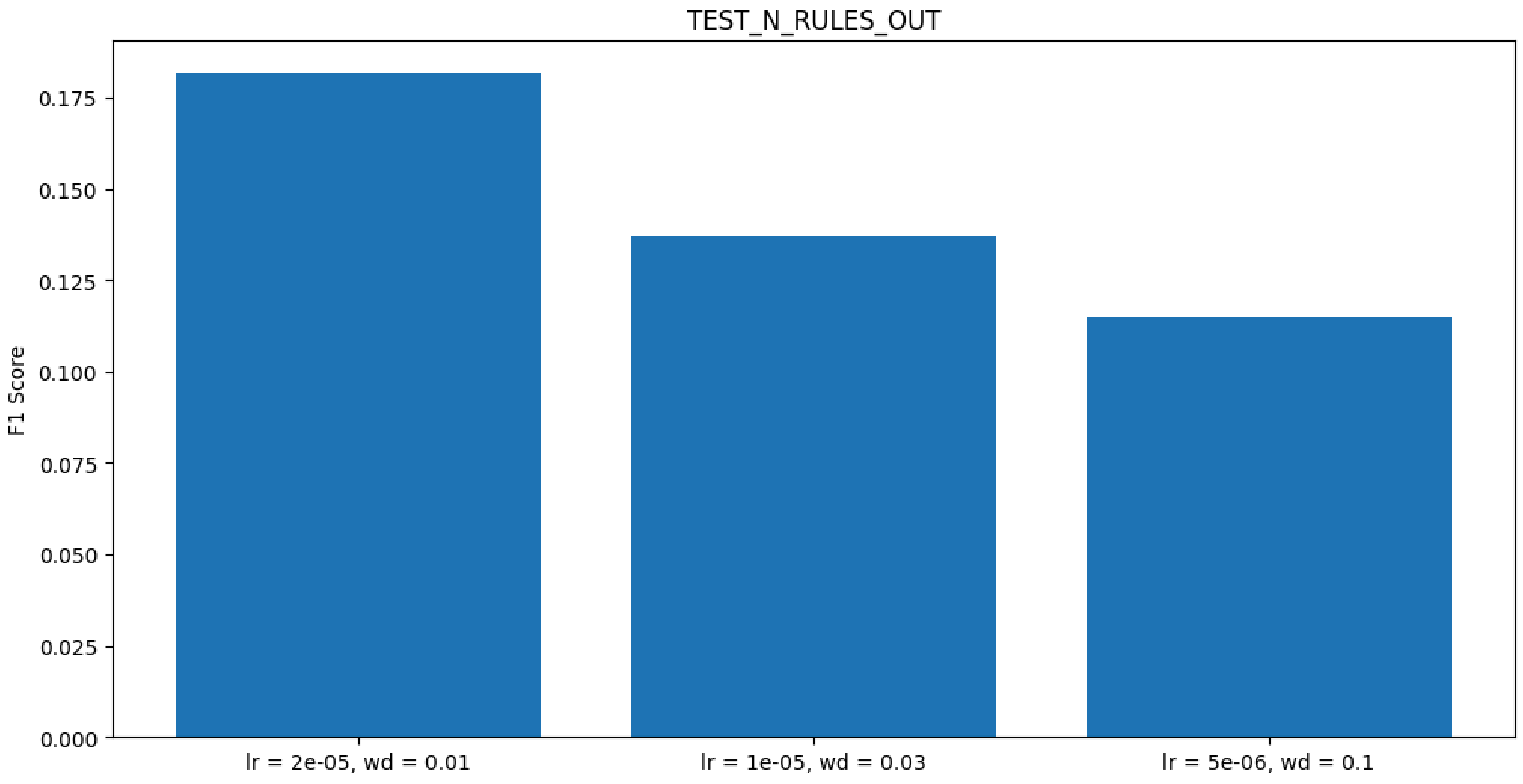
- lr = 2e-5 and wd = 0.01
- lr = 1e - 5 and wd = 0.03
- lr = 5e-6 and wd = 0.1

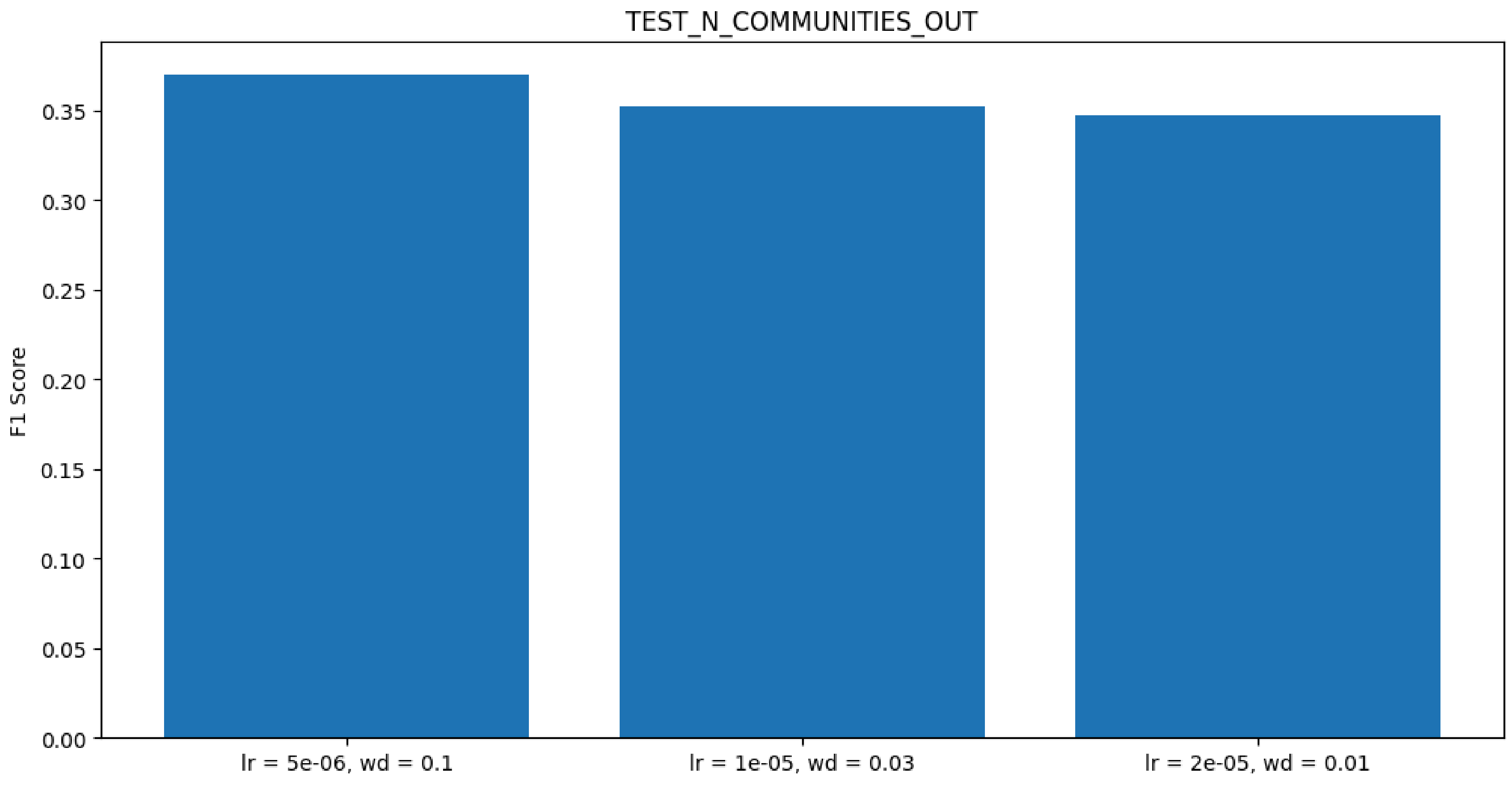
But the best one seems to be lr = 1e - 5 and wd = 0.03

So I used these three parameter combinations to test on unseen rules and communities

I PUT FINAL THREE PARAMETER COMBINATIONS ON OTHER TWO DATASETS

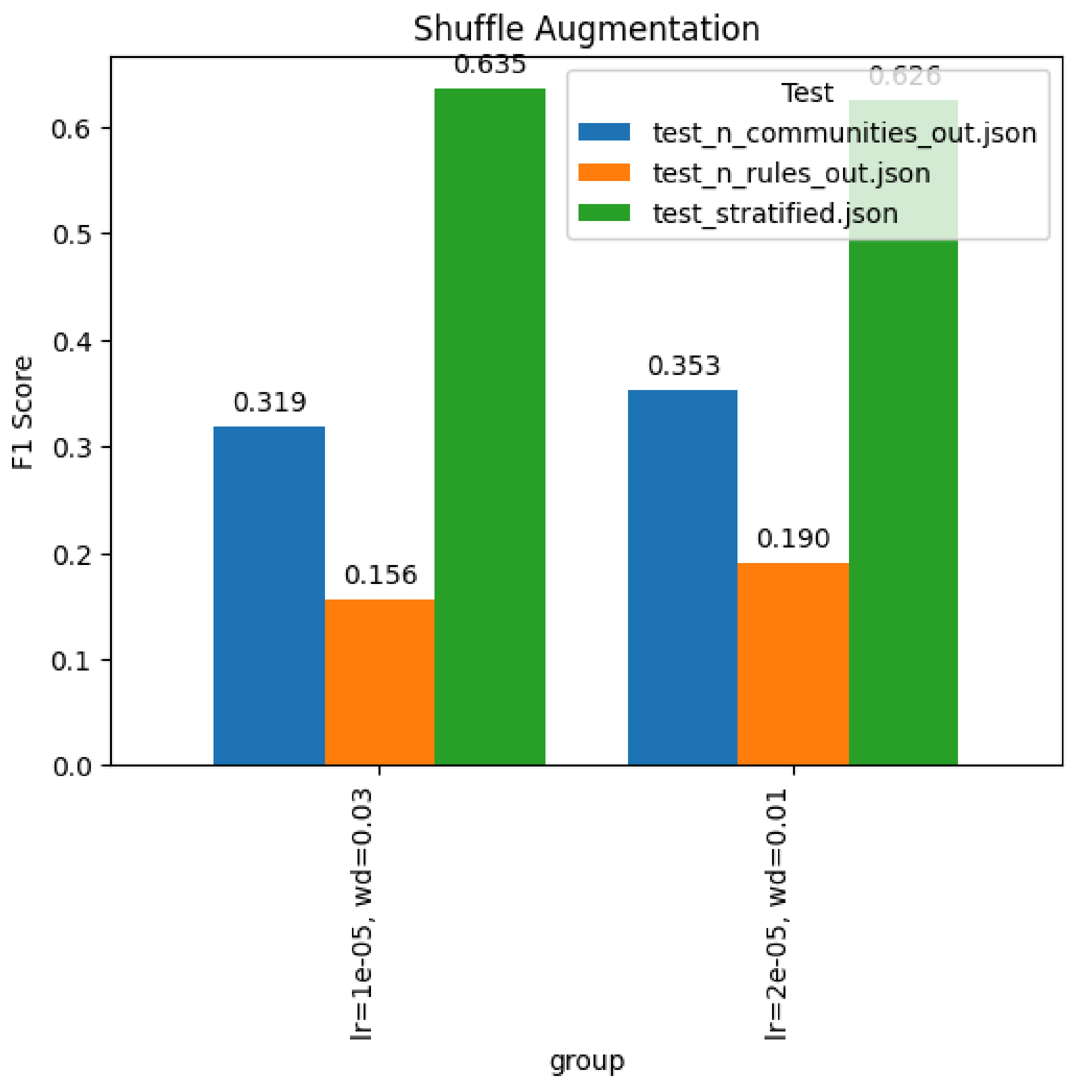
The scores are too small, especially for unseen rules. But better than a dummy model!
The combination of lr = 2e-05 and wd=0.01 looks to perform the best on unseen rules, while for unseen communities, all perform somewhat similarly.
But I decided to exclude the combination of 5e-06 and wd=0.1, since it performs the worst.

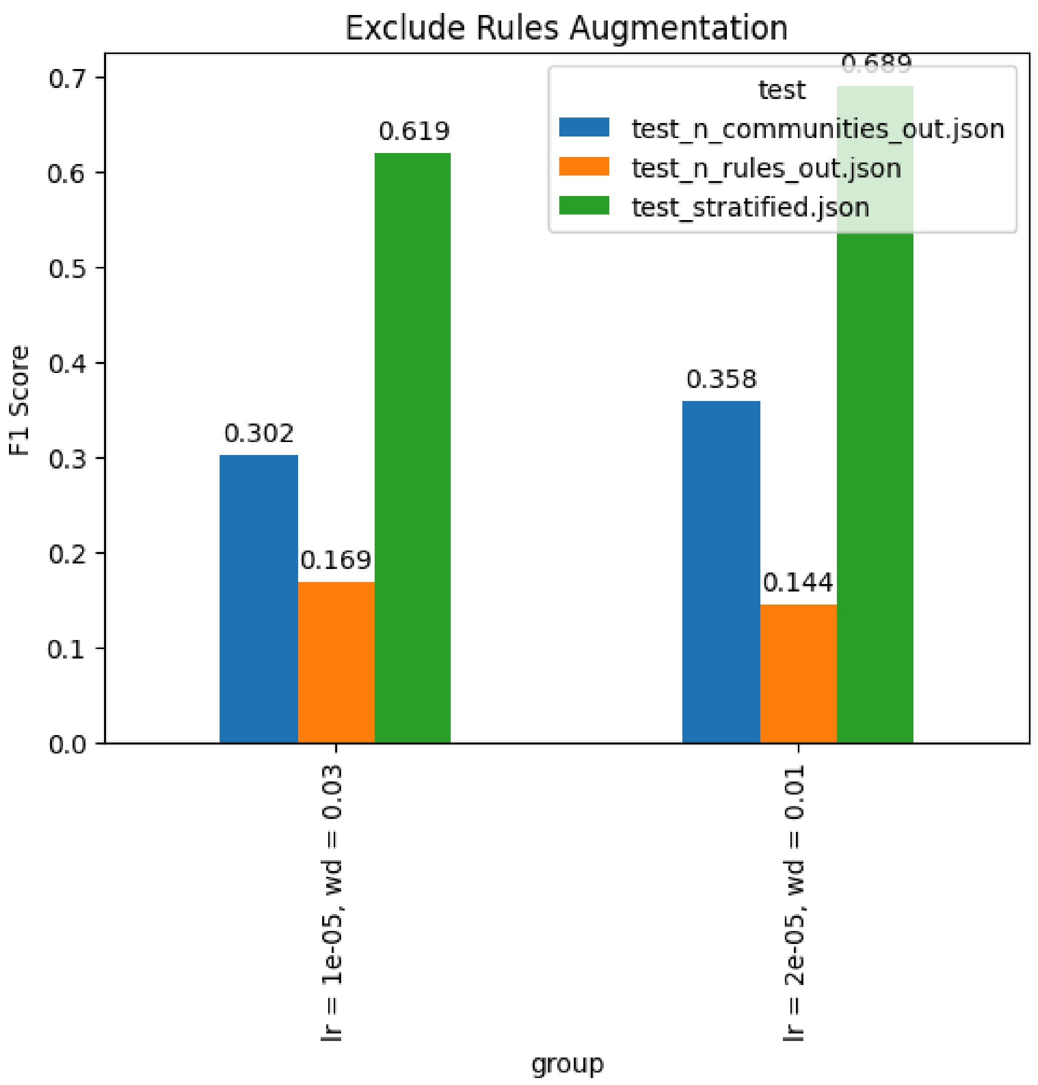


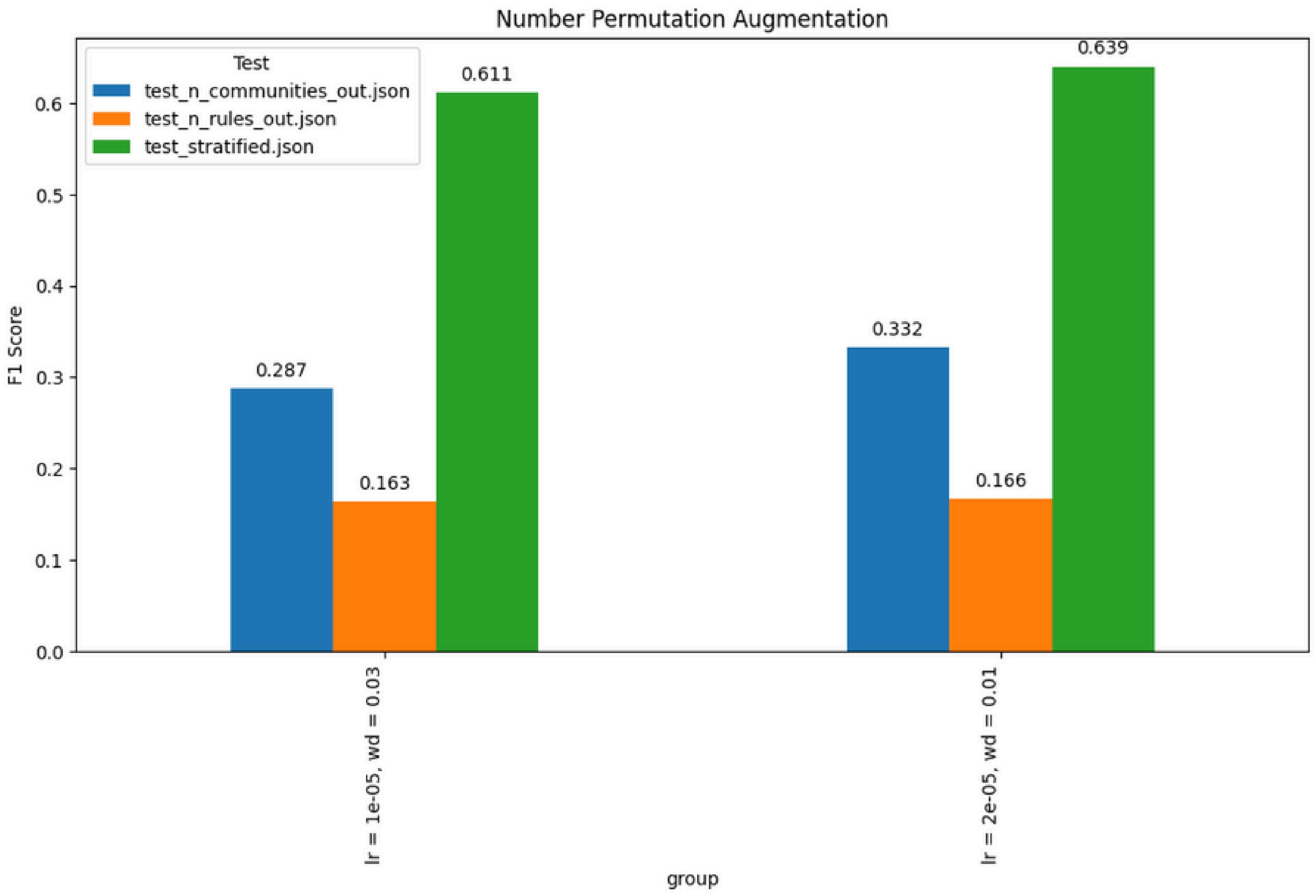


DATA AUGMENTATION

I implemented three data augmentation strategies: rule shuffling, number permutation, and rule exclusion. Initially, I applied each augmentation separately to assess the specific contribution of each method.





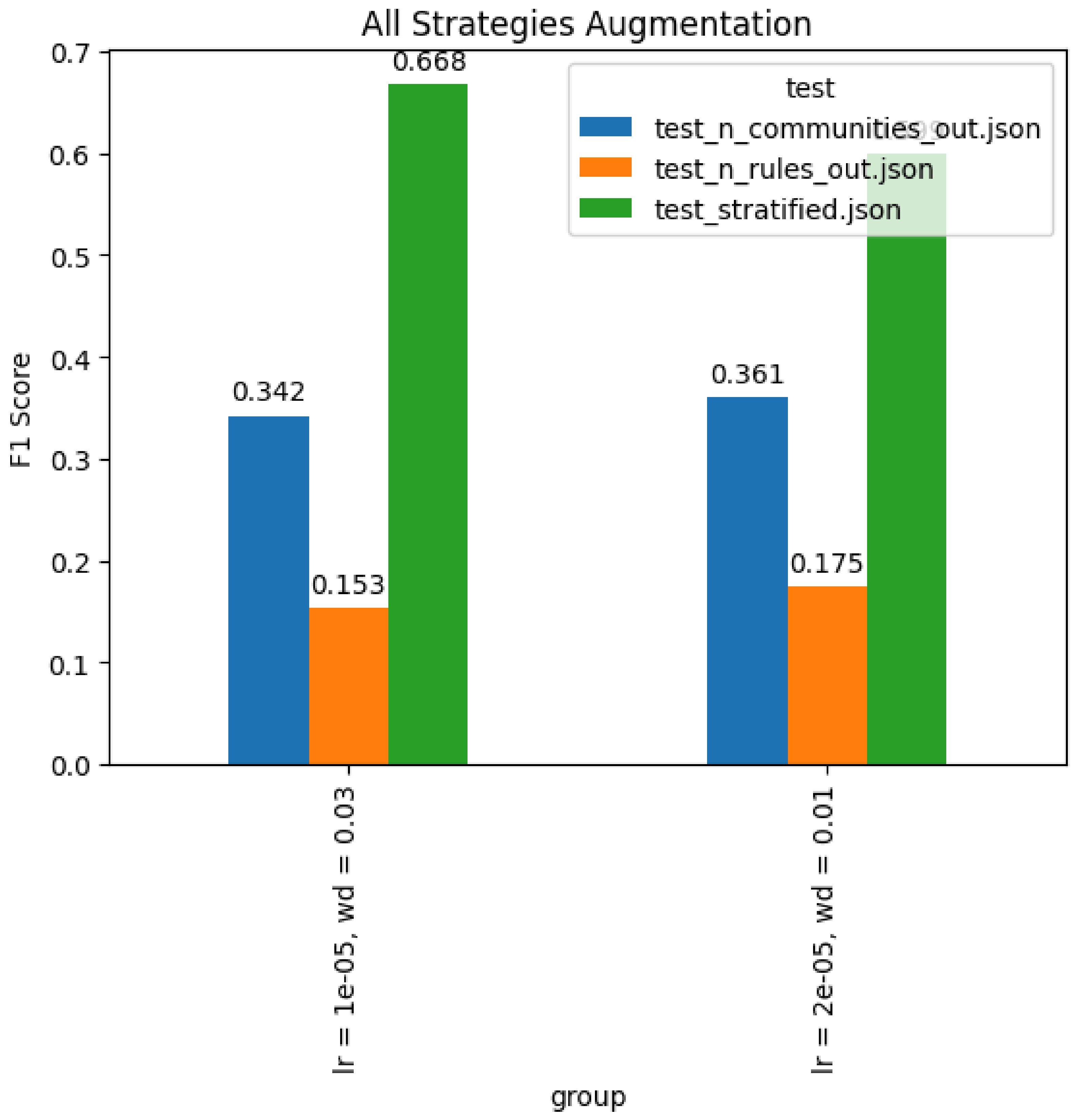


UNSEEN RULES: INCREASED BY 0.01 - 0.015 POINTS
UNSEEN COMMUNITIES: THE SAME

UNSEEN RULES: INCREASED BY 0.03 POINTS FOR LR = 1E-05.
ANOTHER ONE DECREASED BY 0.03 POINTS
UNSEEN COMMUNITIES: DECREASED

UNSEEN RULES: INCREASED BY 0.02 POINTS FOR LR = 1E-05.
ANOTHER ONE DECREASED BY 0.01 POINTS
UNSEEN COMMUNITIES: DECREASED

APPLYING ALL STRATEGIES



LR = 2E-05, WD = 0.01 (FIRST COMBINATION):
UNSEEN RULES - DESCREASED
UNSEEN COMMUNITIES - INCREASED

LR = 1E-05, WD = 0.03 (SECOND COMBINATION):
UNSEEN RULES - INCREASED
UNSEEN COMMUNITIES - THE SAME

The first combination performs worse on unseen rules than applying only the shuffle rule strategy. But for unseen communities, it increases F1 score of raw testing by 0.01 points.

The second combination performs slightly worse on unseen rules than shuffle rule strategy, but better on unseen communities (though the result is the same in comparison to raw testing)

Conclusion: I think it is beteer just to leave shuffle strategy